“We used ChatGPT to review our draft explanations”

IRWA Project Part-3: Ranking & Filtering

Authors: Carla Núñez u214970 / Julia Pérez u214026 / Jan Prats u213927
GitHub URL: https://github.com/Januto30/SearchEngine-IRWA_2025
TAG: IRWA-2025-part-3

After receiving feedback on the previous sections of the project, we understood that, in order to evaluate the performance of the search systems, it is necessary to use the validation_labels document provided by the professors. For the evaluation to be meaningful, it is essential to use queries whose results from our search systems include documents present in validation_labels. Therefore, for this part of the project, it was necessary to modify the queries.
The queries we are going to use in the rest of the project are:

- *“Slim Men Dark Blue Jeans”*
- *“Tapered Fit Blue Jeans”*
- *“Super Skinny Women Blue Jeans”*
- *“Women Blue Jeans”*
- *“Slim Men Blue Jeans”*

1. Explain how the ranking differs when using TF-IDF and BM25, and think about the pros and cons of using each of them. Regarding your own score, justify the choice of the score (pros and cons).

In this part of the project, we experimented with different ranking algorithms that can be applied in a search engine. Our task was to design and implement a retrieval pipeline that:

- Takes a query as input (a piece of text).
- Finds all documents that contain all query terms (conjunctive query, i.e., AND semantics).
- Sorts the matching documents by relevance using different ranking methods.

For doing this possible we implemented 3 different ways of ranking:

a. TF-IDF + cosine similarity: Classical scoring we have seen during the practical labs.

Formula taken into account to implement this algorithm:

$$score(document) = \sum_{term \in query} tf(term, query) * idf(term)$$

We used the TF-IDF vector-space model, without normalizing by vector length (we approximated cosine similarity by summing weighted TF-IDF values). To implement the algorithm:

- We created a function to compute the scores for documents following the formula above.

- For each term in the query and if those terms are present in the inverted index we get the list of term frequencies of those terms, we can know them because we have as a parameter of the function the structure 'tf' storing the term frequency values for each term.
- Once we have that, we loop through all posting lists of the terms.
  We only score the document if they are in the allowed document subset introduced as a parameter in the function.
  After that filter, we get the TF value for this term in this document and if it's misaligned or shorter than the previous tf_list by default TF is 0.0.
- We multiply its TF and IDF and the result is added to the document's score.
- We return the scores.

b. BM25

Formula taken into account to compute the BM25 score:

$$score(document) = \sum_{term\ \epsilon\ query} IDF(term) * \frac{tf(term, document)(k_1+1)}{tf(term, document)\ k_1(1 - b + b * \frac{|document|}{avglen})}$$

$$IDF(term) = log(\frac{N - df(term) + 0.5}{df(term) + 0.5} + 1) \qquad avglen = \frac{total\ number\ of\ words\ in\ all\ documents}{number\ of\ documents}$$

Where:
- $tf(term,\ document)$ = term frequency in document
- $|document|$ = document length
- N = total number of documents in the corpus
- $df(term)$ = document frequency of that term

To implement the algorithm:
- Due to the feedback received in the previous deliverables of the project, we created a function to recompute the document lengths and the average length of all documents.
  To compute the document lengths we revised for every term present in the inverted index how many times is present in each document. So, if we sum all the term frequencies present in each document we will get their length. For computing the average length of all documents we simply used np.mean of all values obtained for document lengths.
- Following the formula to compute the BM25 score, we firstly created a function to compute the IDF function by definition.
- Then we proceed creating the function to score the documents with the BM25 formula. To do that we took into account both previous functions and assigned to each document its score based on the result of the formula defined above to score documents.
- Finally, we created the function bm25_search to impose the condition of conjunctive AND semantics and to rank the documents that follow it and need to be retrieved to the user.

By exploring and comparing both relevance scoring approaches explained above, we were able to analyze how different algorithms affect the ranking order of documents.

Looking at the pros and cons of using each of the ranking algorithms exposed:

Ranking algorithm	**Pros**	**Cons**
TF-IDF + cosine similarity	simple, intuitive	no length normalization: long documents get unfair advantage no TF saturation
BM25	best general ranking	more complex TF saturation: extra repetitions give diminishing returns

Our Score: For the third ranking method, we were asked to design our own scoring algorithm. Instead of modifying TF-IDF or BM25 directly, we created a hybrid ranking function that starts from BM25 (as a strong lexical baseline) and then boosts the score using product-specific numerical attributes from the dataset. The idea is to incorporate meaningful metadata that a typical user would consider relevant when searching for products, such as price, discount, availability, and ratings.

Our algorithm follows two steps:

1. We reuse the BM25 score computed previously, ensuring robust term-based ranking that accounts for document length and TF saturation.

2. For every document that matched the query, we compute a boost factor between 0.1 and 3.0, based on the following normalized product attributes:
    - **average rating** (normalized 0–1): Higher-rated products should rank higher.
    - **in-stock status** (binary 0 or 1): Products that cannot be purchased should not appear above available ones.
    - **price** (normalized by maximum price in the dataset): Cheaper products receive a mild positive boost.
    - **discount** (normalized 0–1): Larger promotions make the item more attractive.

Each feature contributes with a fixed weight:

- rating = 0.35
- in-stock = 0.25
- price = 0.25
- discount = 0.15

The final score is:

$final\_score(d) = BM25(d) * boost(d)$ where,

$boost(d) = 1 + 0.35 \cdot rating\_norm + 0.25 \cdot in\_stock + 0.25 \cdot (1 - price\_norm) + 0.15 \cdot discount\_norm$

Ranking algorithm	Pros	Cons
Our score	Incorporates real product attributes (rating, price, discount, stock) into ranking. More aligned with real-world e-commerce search. Penalizes unavailable or overpriced product.s Extends BM25 while keeping lexical relevance.	Requires clean and complete numerical fields; Boost may dominate BM25 when metadata is noisy Hyperparameters (weights, normalization) must be tuned Results depend heavily on data quality

Finally, to evaluate all ranking algorithms we made a function (evaluate_all_results) that assesses the performance of each method across all test queries. For every query and ranking approach (TF-IDF, BM25, and our custom score), the function compares the top-k retrieved documents against the ground truth relevance labels. It computes Precision@K, which measures the proportion of retrieved documents that are relevant, and Recall@K, which indicates how effectively the method retrieves all relevant documents. These metrics are calculated for each query and then averaged to provide an overall evaluation, allowing a clear comparison of the effectiveness of the three ranking algorithms.

The results obtained are:

```
=== Evaluation Summary (Average Metrics) ===
+----------+-----------+-----------+
| Method   |   Avg P@K |   Avg R@K |
+==========+===========+===========+
| tfidf    |       0.1 |     0.043 |
+----------+-----------+-----------+
| bm25     |       0.12 |     0.052 |
+----------+-----------+-----------+
| our      |       0.1 |     0.043 |
+----------+-----------+-----------+
```

The results of the ranking evaluation show that BM25 slightly outperforms TF-IDF in terms of both precision and recall, with an average Precision@10 of 0.12 compared to 0.10 for TF-IDF, and an average Recall@10 of 0.052 versus 0.043. This is consistent with the BM25 formula, which incorporates document length normalization and TF saturation, helping to better rank relevant documents even in longer product titles. Our custom scoring method achieved similar average metrics to TF-IDF (Precision@10 0.10, Recall@10 0.043), but it shows improvements for some specific queries like “Tapered Fit Blue Jeans,” where it achieves a higher Precision@10 of 0.20. This indicates that the feature-based boosts in our score (rating, stock status, price, discount) can help prioritize more desirable products, although it may not always improve the retrieval of strictly relevant items. Overall, the results confirm that while classical scoring methods like TF-IDF and BM25 are effective for general relevance, integrating product-specific features can adjust rankings to better match practical e-commerce search goals.

2. word2vec + cosine ranking score

For this part of the project, we built a semantic search system using Word2Vec and cosine similarity. The goal was to represent each product title as a vector, represent each query the same way, and then rank documents by how close they are to the query in vector space.

To train the Word2Vec model, the training function produced 100-dimensional vectors, using a context window of size 5 and keeping all tokens in the vocabulary. Once trained, we embedded each title by averaging the embeddings of every word that appears in the model's vocabulary. If a title contained no known words, the embedding was set to a zero vector. All document embeddings were then combined into a matrix so that cosine similarity could be applied efficiently.

The same embedding approach was used for the queries. Each query was converted into a vector by averaging the word embeddings, and the cosine similarity between the query vector and all document vectors was computed. The top 20 results were then selected and printed.

We tested the search system with five queries:

- *"Slim Men Dark Blue Jeans"*
- *"Tapered Fit Blue Jeans"*
- *"Super Skinny Women Blue Jeans"*
- *"Women Blue Jeans"*
- *"Slim Men Blue Jeans"*

Because these queries match the dataset vocabulary closely and include well-represented clothing descriptors (e.g., *slim*, *tapered*, *blue*, *jeans*), the AND-filtering mechanism narrows the search to highly precise subsets. As a result, similarity scores are extremely high (often **0.97 to 1.00**) and retrieved items are perfectly aligned with the query attributes.

- For **"Slim Men Dark Blue Jeans"**, every returned product is an exact title match, and all similarity scores equal **1.000**, indicating that both the filtering and embedding steps produce a perfectly homogeneous result set.

- Although the query **"Tapered Fit Blue Jeans"** does not specify gender, all retrieved items are *Tapered Fit Women Blue Jeans*, each scoring **0.972**. The AND-filter ensures that only products containing *tapered*, *fit*, *blue*, and *jeans* are examined, while the Word2Vec similarity reinforces their grouping.

- For **"Super Skinny Women Blue Jeans"**, the system again returns a full list of exact matches, all with similarity **1.000**. The combination of precise vocabulary and dense representation of denim categories leads to highly uniform embeddings.

- The query **"Women Blue Jeans"** results in a consistent cluster of *Skinny Women Blue Jeans*, with similarity values around **0.969**. Because several skinny-fit variants satisfy all query tokens, they form a coherent subset.

- Finally, **“Slim Men Blue Jeans”** produces another set of perfect matches, all scoring **1.000**, similar to the behavior observed in the first query. This highlights that the dataset contains large, repeated groups of highly similar denim titles, and the search mechanism handles them cleanly.

3. Can you imagine a better representation than word2vec? Justify your answer.

As we described previously, we built a semantic search system using Word2Vec representing each product title as a vector, representing each query the same way, applying the condition of a conjunctive query, i.e., AND semantics. Finally, ranking documents by how close they are to the query in vector space.

To see if there is a better representation than Word2Vec, we built another semantic search system using Doc2Vec. Doing a conceptual comparison:

Ranking algorithm	**Pros**	**Cons**
Word2Vec	fast, simple	lacks context, does not handle phrases
Doc2Vec	captures global context	slower, variable quality

Doc2Vec is a semantic search system that generates vectors for entire documents. To implement Doc2Vec with AND semantics, it was necessary to prepare the documents, train the model, and implement a search function that respects AND semantics.

For document preparation, each document is represented as a TaggedDocument, with ‘words’ = list of tokens and ‘tags’ = unique product identifier (pid). We used the preprocessed texts (‘title_clean’, ‘description_clean’, ‘product_details_text’) so that the AND filter works correctly.

When training the model, Doc2Vec learns vectors for each document, capturing the global semantics of the title + description + details. The main parameters are:

- vector_size: embedding dimension (100)
- window: context size
- min_count: ignores rare words
- epochs: number of iterations to train the vectors effectively
- workers: number of threads used to train the model in parallel

Finally, for the search function implementation:

- The query is preprocessed.
- We filter and keep only the products that satisfy AND semantics.
- We call the function query_to_vector(), which obtains an embedding vector for a text that was not seen during training, as is the case for the query text. This function returns the final vector representing the query.

- We calculate the cosine similarity between the query vector and the filtered document vectors.
- Finally, we return the most semantically similar documents.

To evaluate whether Doc2Vec is better or not we took advantage of the 'evaluate_all_results()' to compare the precision and recall of both semantic search systems. This function evaluates computes Precision@K and Recall@K for each query and then averaging these metrics.

- It converts each method's top-k ranked PIDs into relevance labels using labels_df.
- For every query and every method, it calculates Precision@K and Recall@K and stores the results.
- After all queries are processed, it computes the **average Precision@K** and **average Recall@K** per method.
- Finally, it prints a summary table and returns a dictionary with the averaged metrics.

The results obtained are:

```
=== Evaluation Summary (Average Metrics) ===
+----------+-----------+-----------+
| Method   |   Avg P@K |   Avg R@K |
+==========+===========+===========+
| word2vec |      0.04 |     0.017 |
+----------+-----------+-----------+
| doc2vec  |      0.1  |     0.043 |
```

From the table above, we can see that Doc2Vec more than doubles the average precision and recall, showing that it is better suited for product search when combining titles, descriptions, and details.

The results make sense because, as we know, Word2Vec uses word embeddings and therefore loses context, meaning that identical titles cannot be distinguished semantically.

In contrast, Doc2Vec learns vectors for entire documents, capturing relationships between words and the overall context.